02

Part 2 · Storage, Platforms & Architecture

Where analytical data lives: warehouses, lakes, lakehouses, open formats, engines, and the organisational patterns around them.

- Data Warehouse · Lake · Lakehouse
- Parquet
- Open Table Formats · Delta · Transaction Log
- Apache Spark · Trino
- Lakebase
- Data Mart · Mesh · Fabric
- Medallion (Bronze, Silver, Gold)

2.1 Data Warehouse

★★★

IN ONE LINE

A central store of integrated, historical, structured data modelled for fast analytical queries.

A data warehouse is a central, **modelled** repository of cleaned, historical business data optimised for analytics. Data is transformed *before* it is loaded (**schema-on-write**), so users get consistent, fast SQL on trustworthy tables.

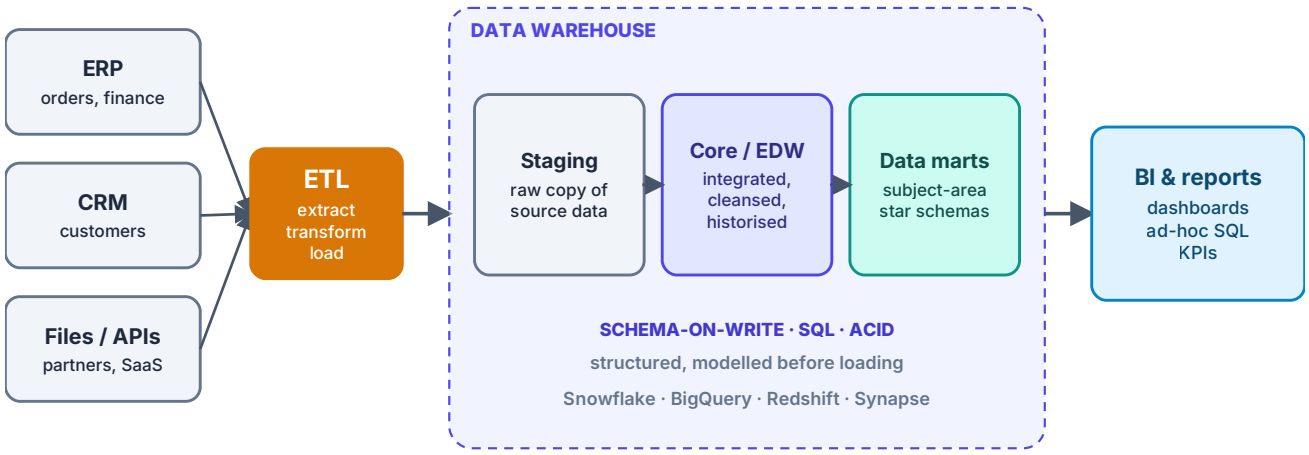


Figure 2.1 — Classic data warehouse architecture

Inmon's four traits

- **Subject-oriented** — organised by business topic (sales), not by app.
- **Integrated** — one consistent version of codes, units, keys.
- **Time-variant** — keeps history for trend analysis.
- **Non-volatile** — data is added, not overwritten by day-to-day ops.

Two schools

- **Inmon (top-down)** — build a normalised enterprise warehouse first, then derive marts from it.
- **Kimball (bottom-up)** — build dimensional marts with conformed dimensions (the "bus"); the union is the warehouse.

COMMON TRAP

Classic warehouses couple storage and compute, store mostly structured data and can be expensive or proprietary. That limitation is exactly why data lakes appeared — and why lakehouses merged the two ideas.

2.2 Data Lake

★★★

IN ONE LINE

Cheap, scalable storage that holds raw data in its native format, with structure applied when read.

A data lake stores **raw data of any type** in cheap object storage with no upfront schema (**schema-on-read**). It is flexible and scales to petabytes, and is the natural home for logs, clickstreams, images and ML training data.

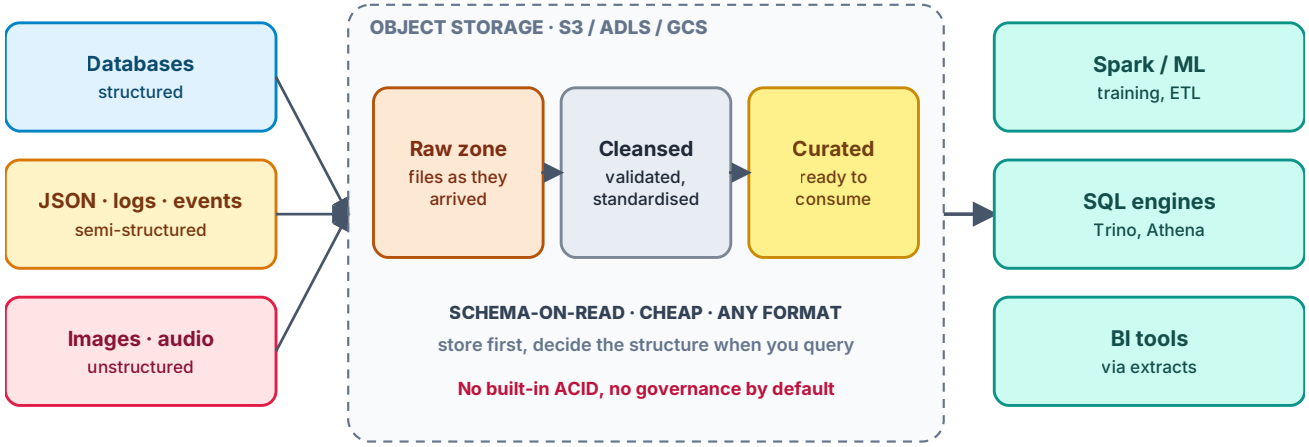


Figure 2.2 — Data lake: land everything, structure later

Strengths

- Very low cost per TB.
- Stores structured, semi- and unstructured data.
- Decoupled storage and compute.
- Ideal for data science and ML.

Weaknesses

- No ACID: failed writes leave partial files.
- No updates / deletes without rewriting files.
- Weak governance, quality and discoverability.
- Small-file and performance problems.

COMMON TRAP

Data swamp — a lake with no catalog, ownership, quality checks or naming standards becomes a dumping ground nobody trusts. The fix is metadata, governance and a layered design (see Medallion).

2.3 Data Lakehouse

★★★

IN ONE LINE

One open architecture combining the low-cost flexibility of a lake with the reliability and speed of a warehouse.

A lakehouse puts **warehouse-grade reliability and SQL performance on top of lake storage** using an open table format. You keep one copy of the data in open Parquet files and get transactions, governance and BI speed — instead of copying data between a lake and a warehouse.

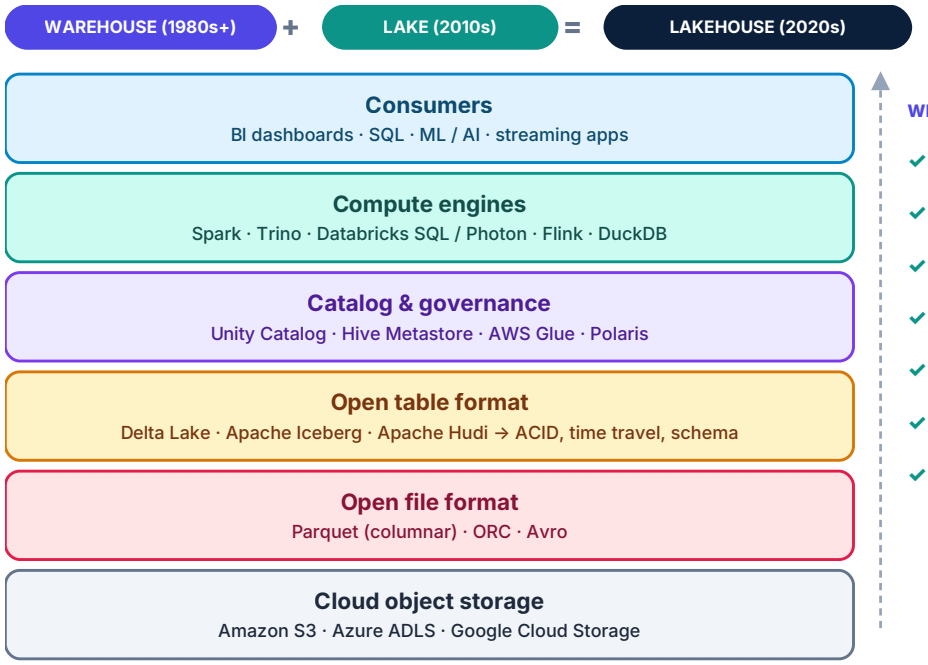


Figure 2.3 — The layered lakehouse stack

	Data warehouse	Data lake	Data lakehouse
Data types	Structured	All types	All types
Schema	On write (strict)	On read (flexible)	Enforced *and* evolvable
ACID	Yes	No	Yes (table format)
Cost / scale	Higher, proprietary storage	Lowest, object storage	Low, object storage
BI / SQL	Excellent	Weak	Excellent
ML / data science	Limited	Excellent	Excellent
Governance	Strong	Weak	Strong (catalog)

INTERVIEW ANGLE

A crisp answer to **"why lakehouse?"**: *it removes the two-tier lake-plus-warehouse architecture, which duplicated data, created stale copies and needed fragile ETL between them.* The enabling technology is the open table format with a transaction log.

2.4 Parquet (Columnar File Format)

★★★

IN ONE LINE

A compressed, columnar, self-describing file format designed for fast analytical reads.

Parquet is the de-facto **columnar file format** of the big-data world. A file is split into **row groups**; inside each, every column is stored contiguously as a **column chunk** made of compressed pages. A **footer** holds the schema and per-chunk statistics, so engines can skip data they do not need.

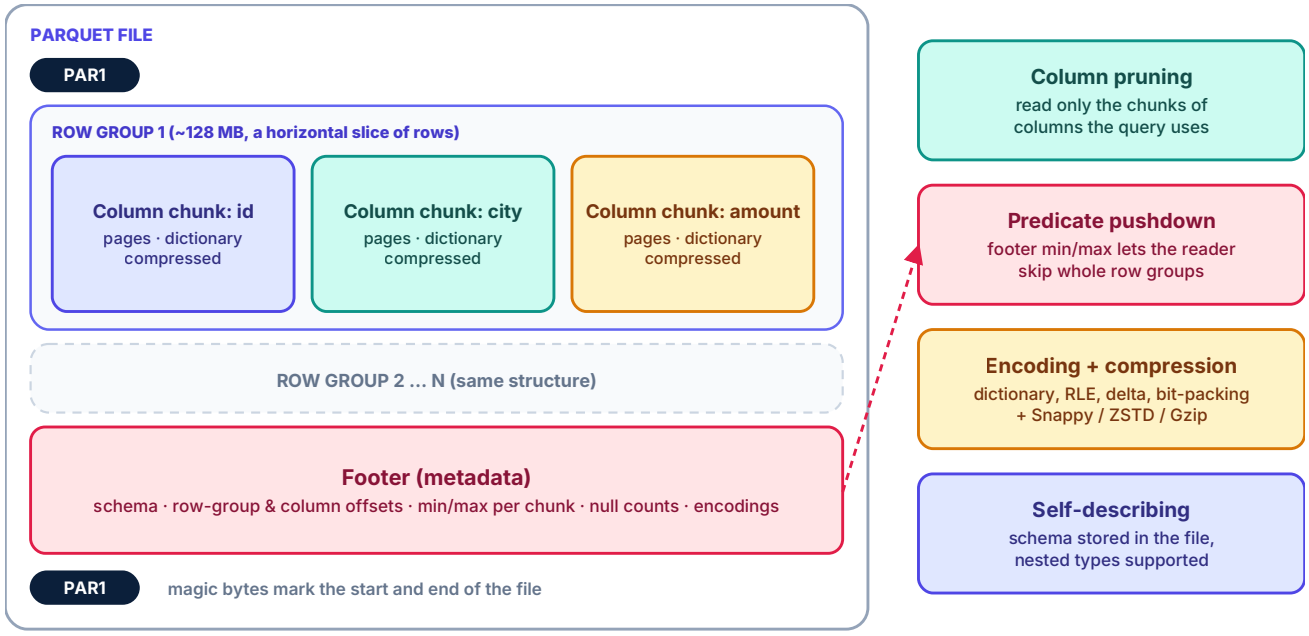


Figure 2.4 — Anatomy of a Parquet file

Format	Layout	Schema	Best for
CSV	Row, text	None (inferred)	Simple exchange; human readable; slow and ambiguous
JSON	Row, text	Implicit, nested	APIs, logs, semi-structured landing data
Avro	Row, binary	Embedded, evolvable	**Streaming / Kafka** messages, write-heavy ingestion
ORC	Columnar, binary	Embedded	Hive ecosystem, strong compression
Parquet	Columnar, binary	Embedded, nested	**Analytics**, lakehouse default

INTERVIEW ANGLE

Common follow-up: *"Parquet vs Delta?"* Parquet is just a **file format**; Delta is a **table format** = Parquet files *plus* a transaction log. Parquet files alone cannot give ACID, time travel or safe concurrent writes. Also be ready to explain that **many tiny Parquet files hurt performance** (metadata overhead, too many tasks).

2.5 Open Table Formats, Delta Lake & the Transaction Log ★★★

IN ONE LINE

A metadata layer (usually a log) that gives ACID tables, time travel and safe concurrency on top of plain Parquet files.

An **open table format** is a metadata layer that turns a pile of immutable files into a **table**: it tracks which files belong to the table at each point in time, what the schema is, and gives transactions on object storage. The three major formats are **Delta Lake**, **Apache Iceberg** and **Apache Hudi**.

Delta Lake & the transaction log

A Delta table is a directory of Parquet files plus a `_delta_log` folder. Every change (INSERT, MERGE, OPTIMIZE ...) appends a numbered JSON **commit** listing `add` and `remove` file actions. The current state is simply *the replay of the log*. Every ~10 commits a **checkpoint** Parquet file summarises the state so readers need not replay thousands of JSON files.

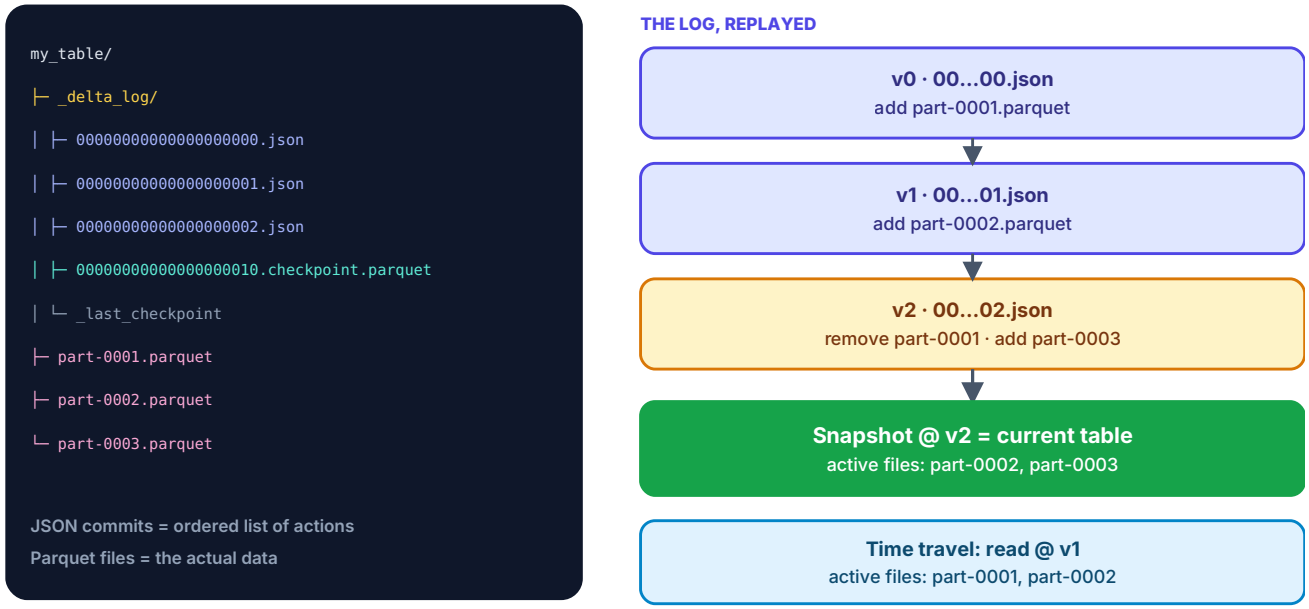
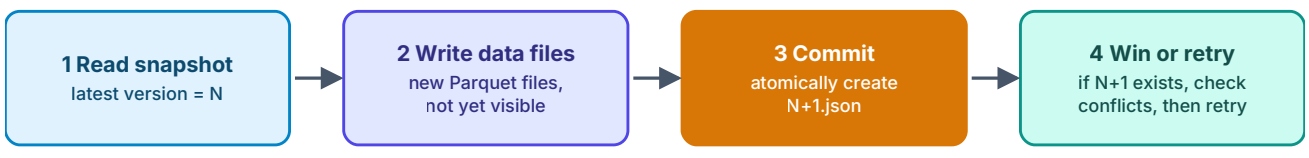


Figure 2.5 — A Delta table on disk and how the log defines each version



Optimistic concurrency: writers do not lock; the first to create the next log file wins.

Readers always see a consistent snapshot, so reads never block writes.

Figure 2.6 — How a Delta write commits atomically (optimistic concurrency)

What the log enables

- **ACID** on object storage — readers only trust files named in a committed version.
- **Time travel** — query or restore any earlier version.
- **Schema enforcement & evolution.**
- **Efficient upsert / delete** with `MERGE`.
- **Change Data Feed** — row-level changes between versions.

Housekeeping commands

- `OPTIMIZE` — compact small files.
- `ZORDER BY` / **liquid clustering** — co-locate related data for skipping.
- `VACUUM` — delete files no longer referenced (breaks time travel past the retention).
- `DESCRIBE HISTORY` — audit the log.

SQL · DELTA LAKE

```sql
-- time travel
SELECT * FROM sales VERSION AS OF 12;
SELECT * FROM sales TIMESTAMP AS OF '2026-10-01';
RESTORE TABLE sales TO VERSION AS OF 12;

-- maintenance
OPTIMIZE sales ZORDER BY (customer_id);
VACUUM sales RETAIN 168 HOURS;
DESCRIBE HISTORY sales;
```

	Delta Lake	Apache Iceberg	Apache Hudi
Metadata	JSON log + Parquet checkpoints	metadata.json → manifest lists → manifests	Timeline of commits + metadata table
Origin	Databricks	Netflix	Uber
Known for	Tight Spark / Databricks integration, MERGE, CDF	Engine-neutral, hidden partitioning, partition evolution	Upserts, incremental pulls, merge-on-read
Interop	UniForm can expose Delta tables to Iceberg readers	Wide engine and catalog (REST) support	Spark, Flink, Presto / Trino

COMMON TRAP

Never run `VACUUM` with a tiny retention (e.g. 0 hours) while long-running readers or streams are active — they may need files you just deleted. The default 7-day retention exists for a reason.

2.6 Apache Spark

★★★

IN ONE LINE

A distributed engine that processes large datasets in parallel across a cluster, with lazy, optimised execution.

Apache Spark is a **distributed, in-memory data-processing engine** for batch, streaming, SQL and ML. You describe *what* you want with DataFrames / SQL; Spark builds an optimised plan and runs it in parallel across a cluster.

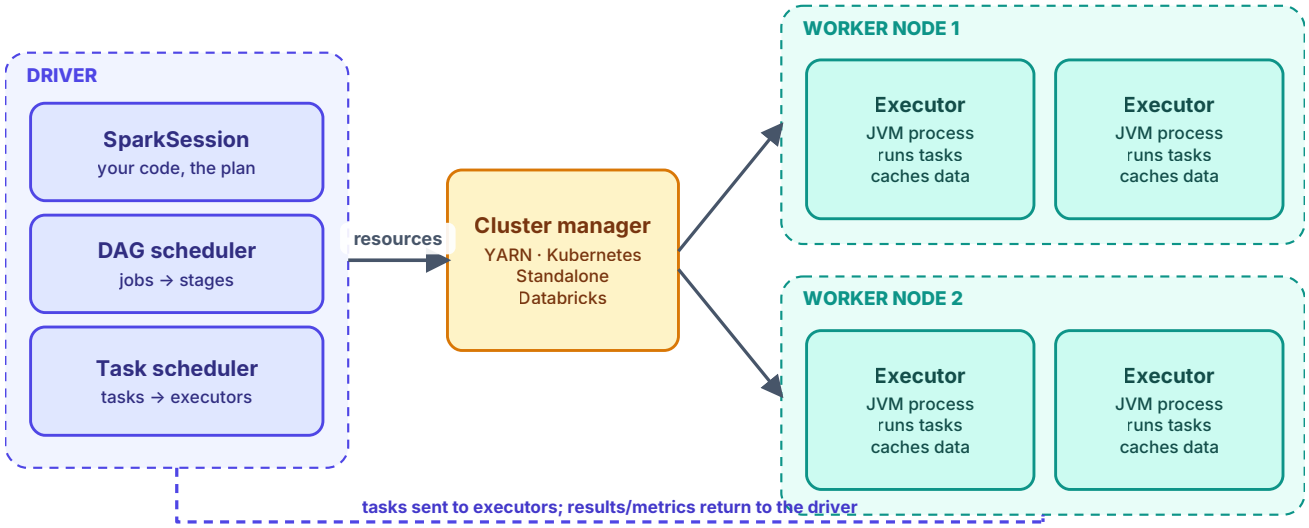
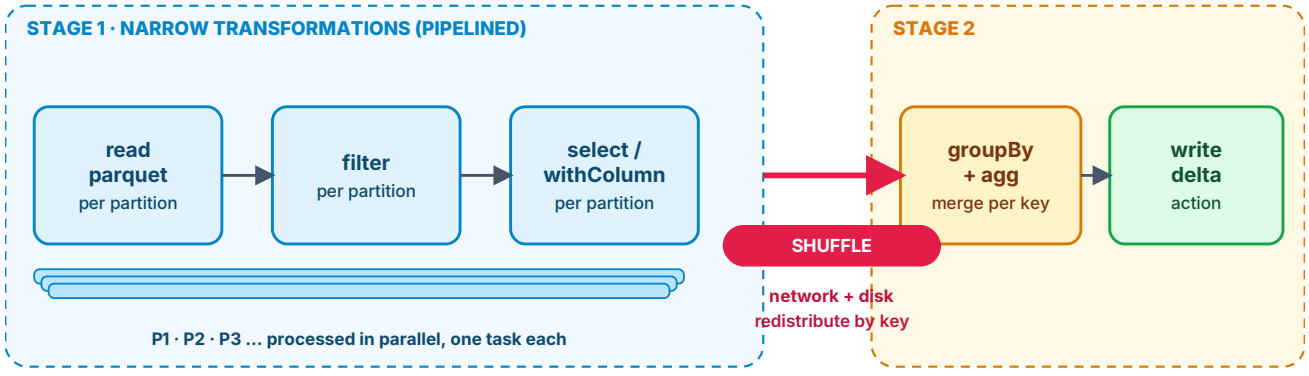


Figure 2.7 — Spark architecture: driver, cluster manager, executors



An ACTION (write / count / collect) triggers the whole DAG; a shuffle is the boundary between stages.

Figure 2.8 — From code to stages and tasks

Concept	Meaning
Lazy evaluation	**Transformations** (select, filter, join, groupBy) only build a plan. **Actions** (count, collect, write, show) trigger execution.
Narrow vs wide	Narrow: each output partition depends on one input partition (filter, map) — no data movement. Wide: needs a **shuffle** (groupBy, join, distinct, orderBy).
Job → Stage → Task	An action creates a job; shuffles split it into stages; each stage runs one **task per partition**.
Catalyst & Tungsten	Catalyst optimises the logical plan (predicate pushdown, column pruning, join reordering); Tungsten generates efficient code and manages memory.
AQE	**Adaptive Query Execution** re-optimises at runtime: coalesces shuffle partitions, switches join strategy, splits skewed partitions.
Partitions	The unit of parallelism. `repartition(n)` shuffles to n; `coalesce(n)` only reduces, without a full shuffle.

PYSPARK

```pyspark
from pyspark.sql import functions as F

df = (spark.read.format('delta').load('/lake/bronze/orders')   # lazy
      .filter(F.col('status') == 'PAID')                      # narrow
      .groupBy('customer_id')                                  # wide -> shuffle
      .agg(F.sum('amount').alias('total')))

df.write.format('delta').mode('overwrite').save('/lake/gold/spend')   # ACTION
```

INTERVIEW ANGLE

The best performance answers are about **shuffles**: broadcast the small side of a join, filter early, avoid Python UDFs (use built-in functions), pick sensible partition counts (`spark.sql.shuffle.partitions` defaults to 200), and fix skew. We expand on this in Part 4.

2.7 Trino

★★☆

IN ONE LINE

A distributed MPP SQL engine that queries data in place across many sources with low latency.

Trino (formerly PrestoSQL) is a **distributed SQL query engine** for fast, interactive analytics. It stores no data itself; **connectors** let it query data where it already lives — lake files, relational databases, Kafka — and even **join across them** in a single query (*federation*).

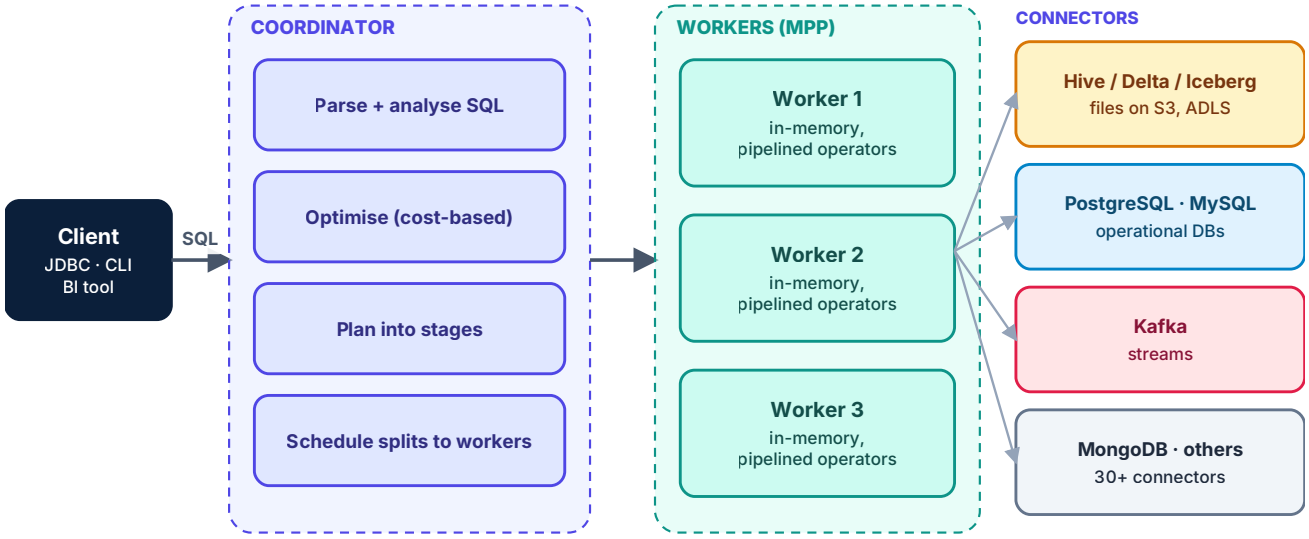


Figure 2.9 — Trino: coordinator, workers and connectors

TRINO · FEDERATED QUERY

```
-- one query, two systems
SELECT c.segment, SUM(o.amount) AS revenue
FROM hive.sales.orders o              -- Parquet on S3
JOIN postgresql.crm.customers c       -- live Postgres table
  ON o.customer_id = c.id
WHERE o.order_date >= DATE '2026-01-01'
GROUP BY c.segment;
```

	Apache Spark	Trino
Primary role	Large-scale ETL / ELT, streaming, ML	Fast interactive SQL, ad-hoc analytics, federation
Execution	Stage-based with shuffles to disk; resilient (task retries)	Pipelined MPP in memory; very low latency
Fault tolerance	Built in (lineage / task retry)	Queries traditionally fail on node loss; optional fault-tolerant mode exists
Languages	SQL, Python, Scala, Java, R	SQL only
Writes	Rich (batch + streaming writers)	Supported via connectors (e.g. Iceberg / Delta), mostly for SQL transforms
Pick it when	Heavy transformations, pipelines, ML	Analysts need seconds-latency SQL across many sources

INTERVIEW ANGLE

A neat one-liner: **"Spark is the workhorse that builds your tables; Trino is the query layer that explores them."** Both read the same Delta / Iceberg tables in a lakehouse.

2.8 Lakebase ★★☆

IN ONE LINE

Databricks' serverless Postgres OLTP database, integrated with the lakehouse so apps and analytics share governed data.

Lakebase is Databricks' **fully managed, Postgres-compatible OLTP database** built into the lakehouse platform. It is based on the serverless Postgres technology from **Neon** (acquired by Databricks in 2025) and separates compute from storage, so it can autoscale, scale to zero and offer database-style *branching*.

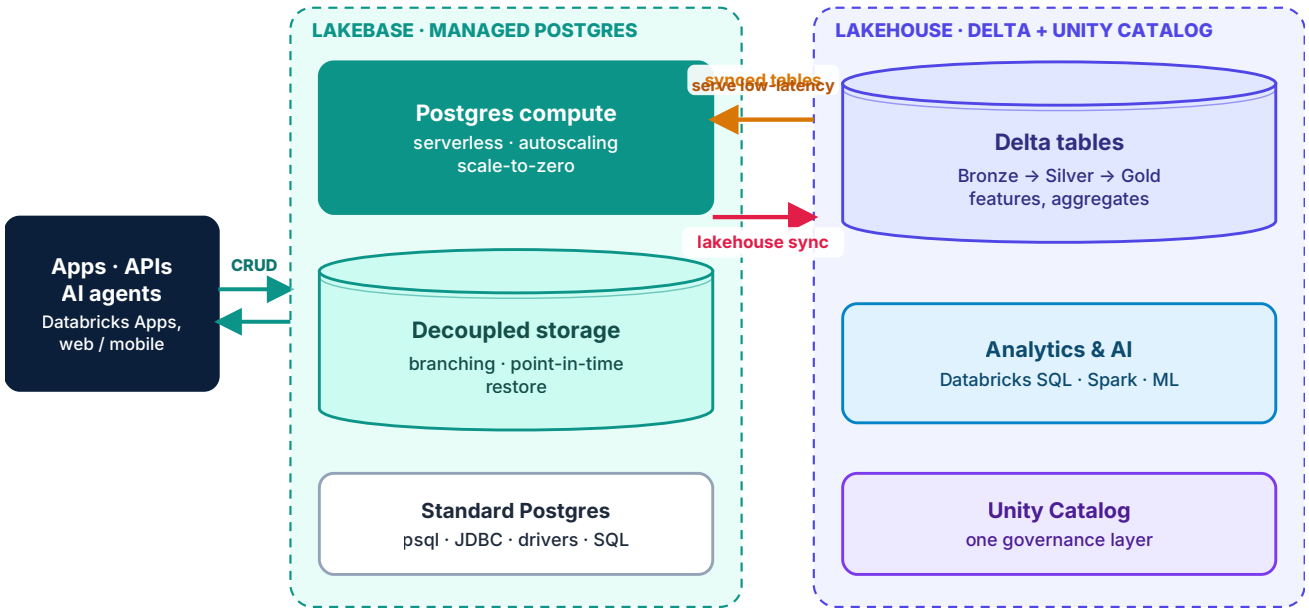


Figure 2.10 — Lakebase sits on the operational side and is wired to the lakehouse

Why it exists

Historically the OLTP database (RDS, Aurora, Cloud SQL) lived *outside* the analytics platform, so teams built fragile ETL to move data in and reverse-ETL to push insights back to apps. Lakebase puts transactions and analytics on one governed platform so both directions become managed sync rather than custom pipelines.

Typical uses

- Operational apps and APIs that need row-level transactions (Databricks Apps).
- Serving lakehouse features / model outputs with low latency (online feature store).
- State for AI agents and RAG applications.
- Replacing a separate managed Postgres next to the lakehouse.

Key ideas to name-drop

- **Synced tables** — a Unity Catalog / Delta table mirrored into Postgres for fast lookups.
- **Lakehouse Sync** — operational data flows back into Delta for analytics.
- **Standard Postgres** — existing drivers and tools work.
- Governed through **Unity Catalog**.

COMMON TRAP

Lakebase is **not** a replacement for your warehouse or lakehouse tables — it handles OLTP workloads next to them. Product names, regions and GA status are still evolving, so check the current Databricks documentation before quoting specifics in an interview.

INTERVIEW ANGLE

Frame it as the industry trend **"converging OLTP and OLAP"** (like HTAP), but done by *integration* (Postgres + lakehouse + sync) rather than a single hybrid engine.

2.9 Data Mart

★★☆

IN ONE LINE

A subject-specific slice of analytical data built for one team or business function.

A data mart is a **focused subset of analytical data for one business function or team** — typically a small star schema. It is smaller, faster to query, easier to secure, and speaks the business unit's own language.

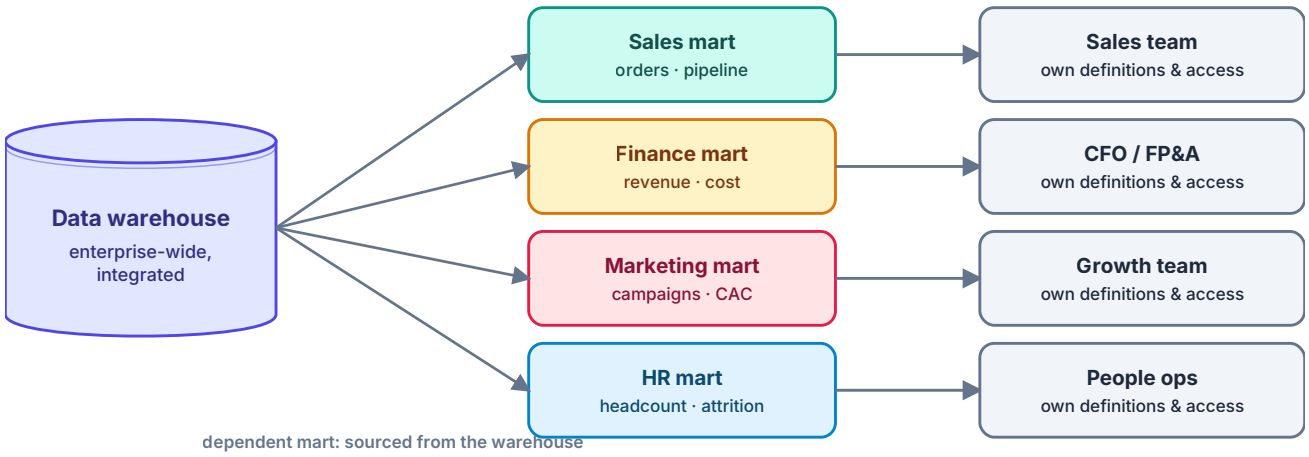


Figure 2.11 — Departmental data marts fed from the central warehouse

	Data warehouse	Data mart
Scope	Enterprise-wide, many subject areas	One department / subject area
Size	Large (TBs–PBs)	Small (GBs–low TBs)
Audience	Analysts, data scientists, org-wide	Business users in one function
Build time	Months	Weeks
Modelling	Normalised core and/or conformed dimensions	Dimensional (star schema)

Dependent mart

Built **from** the warehouse. Consistent numbers, single source of truth. (Inmon style.)

Independent mart

Built straight from sources. Quick, but risks "data silos" and conflicting KPIs unless dimensions are conformed. (Kimball bus keeps this safe.)

REMEMBER
In a lakehouse the **Gold layer is effectively the set of data marts**: curated, business-aligned tables per domain.

2.10 Data Mesh

★★★

IN ONE LINE
A decentralised approach where business domains own and publish data as products on a shared self-serve platform.

Data mesh (Zhamak Dehghani) is a **socio-technical approach** that fixes the bottleneck of a single central data team. Instead of funnelling everything into one pipeline team, data ownership is **decentralised to business domains**, who publish reliable data as products on a shared platform.

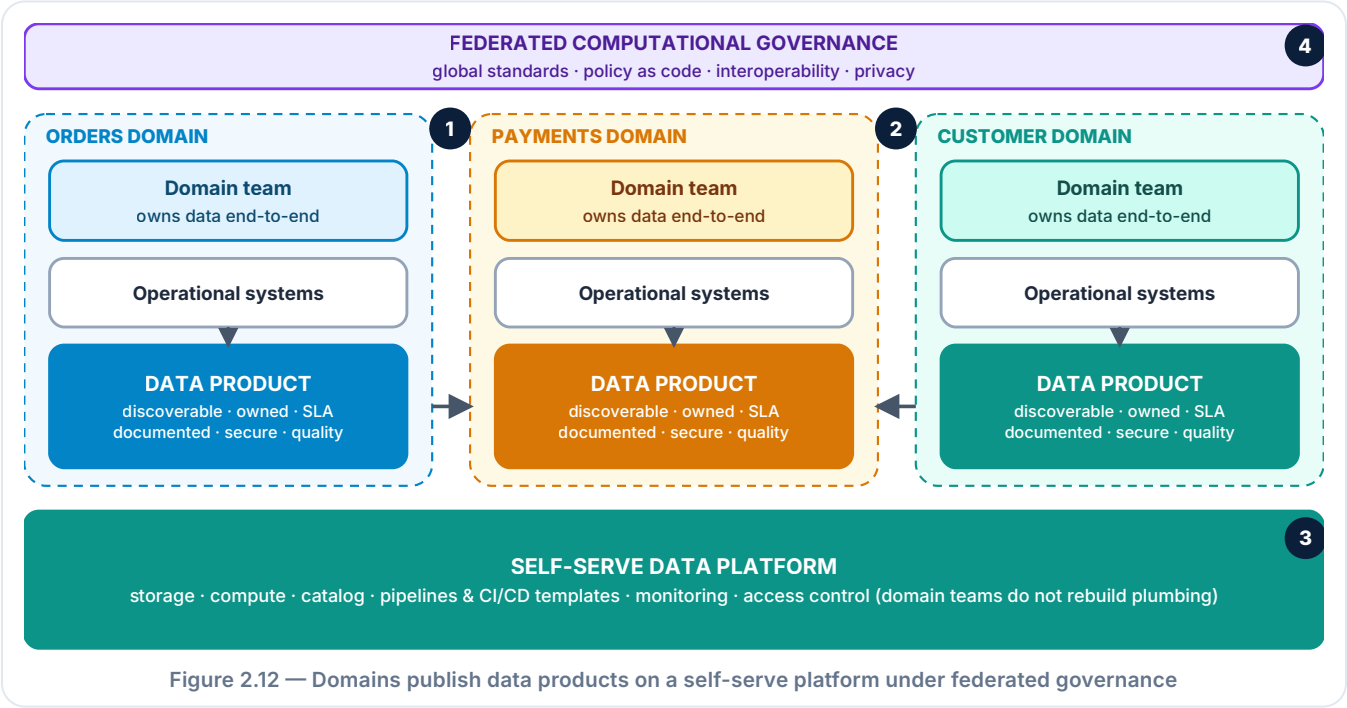


Figure 2.12 — Domains publish data products on a self-serve platform under federated governance

#	Principle	What it means in practice
1	Domain-oriented ownership	The team closest to the data (Orders, Payments…) owns, models and serves it.
2	Data as a product	Datasets have owners, SLAs, documentation, versioning, quality metrics and a consumer-first mindset.
3	Self-serve data platform	A platform team provides paved-road tooling so domains can publish without being infrastructure experts.
4	Federated computational governance	Global rules (security, naming, interoperability) are agreed centrally but **enforced by code** in the platform.

COMMON TRAP
Mesh is an *organisational* pattern, not a product you buy. It adds real overhead and suits large organisations with many autonomous domains; small teams are usually better served by a well-run central lakehouse.

2.11 Data Fabric

★★☆

IN ONE LINE

A metadata-driven, automated integration layer that unifies access to data across many systems.

A data fabric is a **technology architecture** that weaves distributed data sources together with an intelligent, **metadata-driven integration layer**. It continuously collects "active metadata" (lineage, usage, quality) and uses automation / AI to discover, connect, govern and serve data wherever it lives — on-prem, multi-cloud, lake or SaaS.

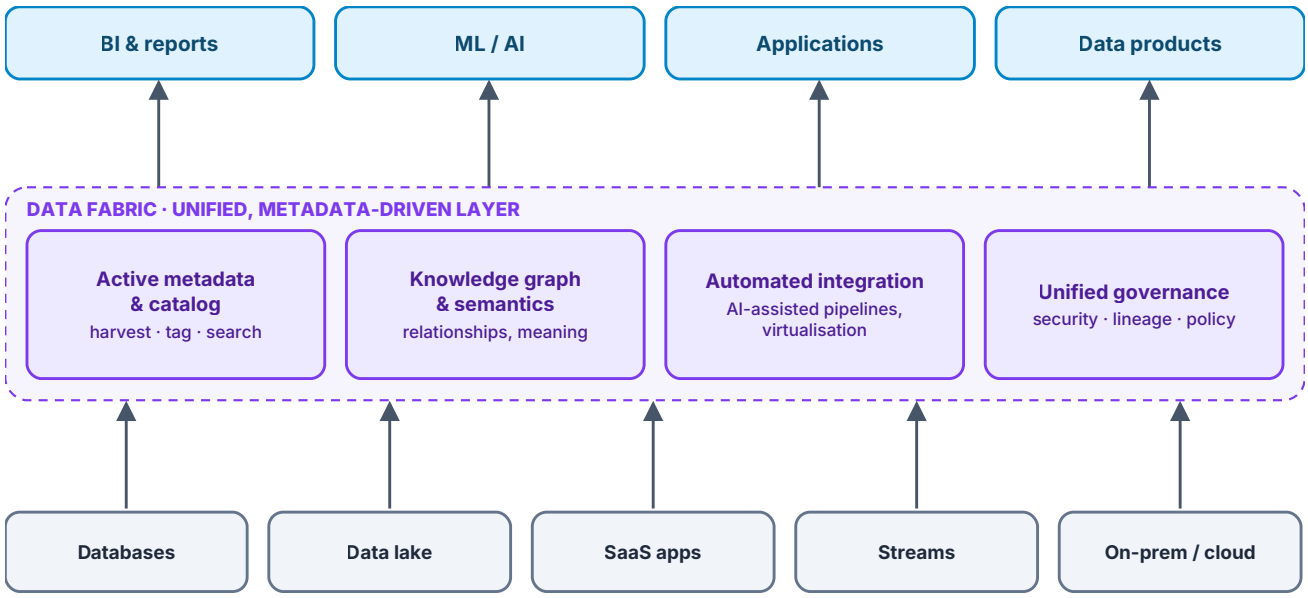


Figure 2.13 — Data fabric: a metadata and automation layer over heterogeneous sources

	Data mesh	Data fabric
Nature	Organisational / cultural	Technological / architectural
Centre of gravity	People and domain ownership	Metadata and automation
Data location	Stays with domains	Stays where it is; virtualised / integrated
Governance	Federated, domain-enforced	Centralised policies applied automatically
Can combine?	Yes — a fabric can be the platform that enables a mesh	

INTERVIEW ANGLE

The summary line: **mesh = "who owns the data", fabric = "how data is connected and automated".** They are complementary, not rivals.

2.12 Medallion Architecture (Bronze, Silver, Gold)

★★★

IN ONE LINE

A layered lakehouse design that progressively refines raw data into trusted, business-ready tables.

Medallion architecture organises a lakehouse into **three progressive quality layers**. Data is never "fixed in place": each hop reads one layer and writes the next, so you can always trace and rebuild downstream tables from the raw data.

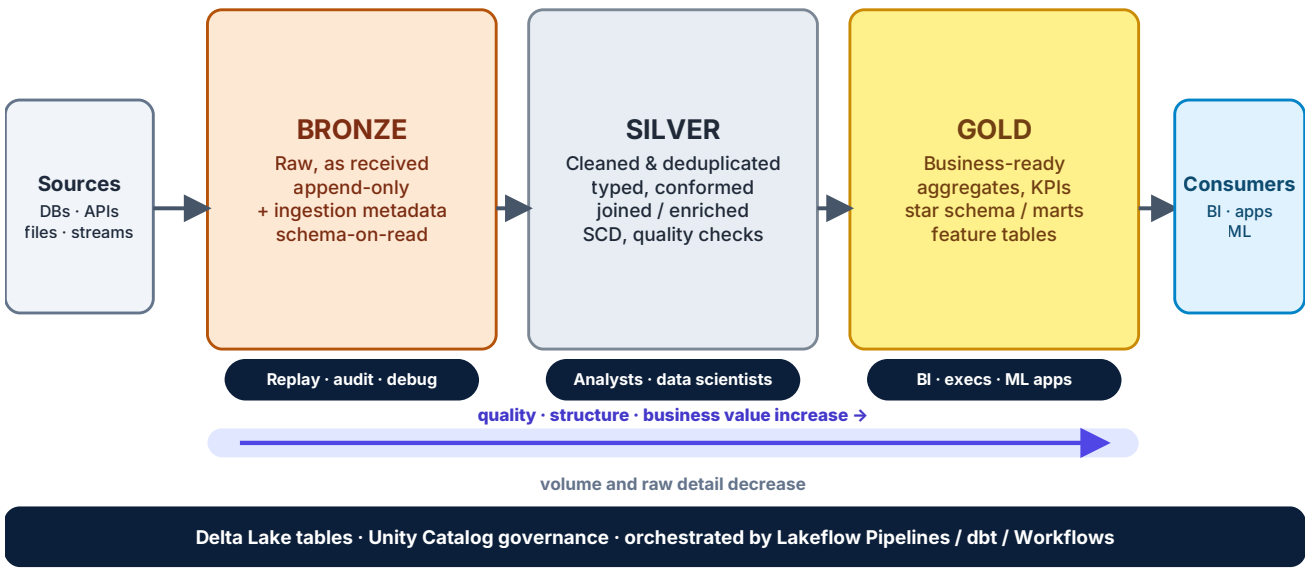


Figure 2.14 — Bronze → Silver → Gold

	Bronze	Silver	Gold
State of data	Raw copy, minimal change	Validated, deduped, conformed	Aggregated, modelled for use
Schema	As source (+ load time, source file)	Enforced, typed, standardised	Business model (star schema, wide tables)
Typical ops	Ingest (Auto Loader, CDC, Kafka)	Cleanse, join, apply CDC, SCD2, DQ rules	Aggregate, KPI logic, semantic definitions
Write pattern	Append	Merge / upsert	Overwrite or incremental merge
Consumers	Data engineers	Analysts, data scientists	BI, executives, ML applications

Why layers?

- Reprocess from Bronze after a bug — no need to re-pull the source.
- Clear data-quality contracts at each stage.
- Different access controls per layer.
- Incremental, restartable pipelines.

Variations

- Add a **quarantine** table for rows that fail checks.
- Some teams say raw / refined / curated, or add a **Platinum** / serving layer.
- Medallion is a *pattern*, not a product — it applies on any lakehouse.

INTERVIEW ANGLE

Interviewers often ask **“where would you deduplicate / apply business rules / mask PII?”**. Standard answers: dedupe and type-cast in **Silver**, business metrics in **Gold**, keep Bronze faithful to the source and mask PII on the way into Silver (or restrict Bronze access).